

LABORATORY ACTIVITY 8

Build an MVVM App: Create & Update a User Profile

Student Handout & Guidelines • Kotlin + Jetpack Compose • Android Studio

Course	_____
Topic	MVVM & ViewModels (Kotlin, Jetpack Compose)
Instructor	Mr. Ernesto Razo Jr.
Points	100
Required branch	lab-activity-8
Student name / section	_____

1. Overview

In this beginner-friendly lab you will build an Android app where a user can create and update a profile using the MVVM (Model–View–ViewModel) pattern. The profile has a name, email, contact number, address, username, and a list of skills the user can add and remove. A Preview button switches to a read-only view of the finished profile.

You will practice the three habits of good MVVM: **all state lives in the ViewModel**, the **UI only shows state and sends events**, and **state flows down while events flow up**. Because the state lives in the ViewModel, everything the user typed survives a screen rotation.

Estimated time: 60–90 minutes.

Go one step at a time and run the app often. If something turns red in Android Studio, press Alt+Enter (Option+Enter on Mac) to add the missing import.

2. Learning Objectives

By the end of this activity you will be able to:

- Create a **ViewModel** in Kotlin that holds the state of a whole form.
- Model a screen's state as a single immutable data class (text fields plus a list).
- Update state immutably with `copy()` as the user types.
- Add and remove items in a list (skills) the correct MVVM way.
- Show different UI (an edit form vs. a preview) based on state.
- Use Git branches — create, switch to, and submit on the **lab-activity-8** branch.

3. Prerequisites

- Android Studio installed (any recent version) with an emulator or a physical Android device.
- A project created from the **Empty Activity** template (this gives you Jetpack Compose already set up).
- Git installed, and your project already tracked in a Git repository (local, plus a remote such as GitHub if your instructor requires pushing).

4. Setup — Get on the Correct Branch

Read this first — it is graded.

All of your work for this activity must be done and submitted on a branch named exactly **lab-activity-8**. Work committed on any other branch (for example `main`) will not be graded.

Open the Terminal inside Android Studio (View ► Tool Windows ► Terminal) and run these commands one by one:

```
# 1) Make sure you are inside your project folder
cd path/to/your/AndroidProject

# 2) Create the required branch AND switch to it
git checkout -b lab-activity-8

# 3) CONFIRM you are on the correct branch (very important!)
git branch
# The current branch is marked with a * -> * lab-activity-8
```

The line with the `*` must read **lab-activity-8**. If it does not, run `git checkout lab-activity-8` and check again before you continue.

5. Build the App (Step by Step)

Step 1 — Add the ViewModel + Compose libraries

Open `app/build.gradle.kts` (Module: app), make sure these lines are inside the `dependencies { }` block, then click **Sync Now**:

```
dependencies {
    val lifecycleVersion = "2.10.0"

    // Lets you call viewModel() inside a Composable
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:$lifecycleVersion")

    // Lets you use collectAsStateWithLifecycle()
    implementation("androidx.lifecycle:lifecycle-runtime-compose:$lifecycleVersion")
}
```

Tip

If a line is highlighted, click “Sync Now” in the yellow bar at the top of the editor. Nothing else will work until the project syncs successfully.

Step 2 — Create the UI state (all the profile data)

Create a new Kotlin file named `ProfileUiState.kt`. This one immutable object holds everything on the screen.

```
data class ProfileUiState(  
    val name: String = "",  
    val email: String = "",  
    val contactNumber: String = "",  
    val address: String = "",  
    val username: String = "",  
    val skills: List<String> = emptyList(),  
    val newSkill: String = "", // text typed in the "add skill" box  
    val isPreview: Boolean = false // false = editing, true = preview  
)
```

The five text fields are simple `Strings`. `skills` is a read-only list, `newSkill` holds whatever the user is currently typing to add, and `isPreview` decides which screen to show.

Step 3 — Create the ViewModel (the brain)

Create a file named `ProfileViewModel.kt`. It holds the state and every action the user can take. It has **no** reference to any screen.

```
import androidx.lifecycle.ViewModel  
import kotlinx.coroutines.flow.MutableStateFlow  
import kotlinx.coroutines.flow.StateFlow  
import kotlinx.coroutines.flow.asStateFlow  
import kotlinx.coroutines.flow.update  
  
class ProfileViewModel : ViewModel() {  
  
    // Private, editable state – only the ViewModel changes it  
    private val _uiState = MutableStateFlow(ProfileUiState())  
    // Public, read-only state – the screen observes this  
    val uiState: StateFlow<ProfileUiState> = _uiState.asStateFlow()  
  
    // --- Update each text field ---  
    fun onNameChange(value: String) =  
        _uiState.update { it.copy(name = value) }  
  
    fun onEmailChange(value: String) =  
        _uiState.update { it.copy(email = value) }  
  
    fun onContactChange(value: String) =  
        _uiState.update { it.copy(contactNumber = value) }
```

```

fun onAddressChange(value: String) =
    _uiState.update { it.copy(address = value) }

fun onUsernameChange(value: String) =
    _uiState.update { it.copy(username = value) }

fun onNewSkillChange(value: String) =
    _uiState.update { it.copy(newSkill = value) }

```

Each `onXChange` makes a new copy of the state with one field changed. Now add the skills and preview actions inside the same class (before the final `}`):

```

// --- Add / remove skills ---
fun addSkill() {
    val skill = _uiState.value.newSkill.trim()
    if (skill.isEmpty()) return // ignore empty input
    _uiState.update { current ->
        current.copy(
            skills = current.skills + skill, // new list = old + one
            newSkill = "" // clear the input box
        )
    }
}

fun removeSkill(skill: String) {
    _uiState.update { current ->
        current.copy(skills = current.skills - skill)
    }
}

// --- Preview toggle ---
fun showPreview() = _uiState.update { it.copy(isPreview = true) }
fun backToEdit() = _uiState.update { it.copy(isPreview = false) }
}

```

Adding a skill builds a brand-new list with `skills + skill`; removing builds one with `skills - skill`. We never edit the old list directly — that is the immutable, MVVM-safe way.

Step 4 — Build the edit form (the View)

Create a file named `ProfileScreen.kt`. Start with the imports and the form composable:

```

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.Button
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier

```

```

import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.lifecycle.compose.collectAsStateWithLifecycle

@Composable
fun ProfileForm(state: ProfileUiState, viewModel: ProfileViewModel) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
            .verticalScroll(rememberScrollState())
    ) {
        Text("My Profile", fontSize = 24.sp,
            fontWeight = FontWeight.Bold)
        Spacer(Modifier.height(12.dp))

        OutlinedTextField(
            value = state.name,
            onValueChange = { viewModel.onNameChange(it) },
            label = { Text("Full name") },
            modifier = Modifier.fillMaxWidth()
        )
        OutlinedTextField(
            value = state.email,
            onValueChange = { viewModel.onEmailChange(it) },
            label = { Text("Email") },
            modifier = Modifier.fillMaxWidth()
        )
        OutlinedTextField(
            value = state.contactNumber,
            onValueChange = { viewModel.onContactChange(it) },
            label = { Text("Contact number") },
            modifier = Modifier.fillMaxWidth()
        )
        OutlinedTextField(
            value = state.address,
            onValueChange = { viewModel.onAddressChange(it) },
            label = { Text("Address") },
            modifier = Modifier.fillMaxWidth()
        )
        OutlinedTextField(
            value = state.username,
            onValueChange = { viewModel.onUsernameChange(it) },
            label = { Text("Username") },
            modifier = Modifier.fillMaxWidth()
        )
    }
}

```

Still inside the same Column, add the skills area and the Preview button:

```

        Spacer(Modifier.height(16.dp))
        Text("Skills", fontWeight = FontWeight.Bold)

        // Type a skill + Add button
        Row(verticalAlignment = Alignment.CenterVertically) {
            OutlinedTextField(

```

```

        value = state.newSkill,
        onValueChange = { viewModel.onNewSkillChange(it) },
        label = { Text("Add a skill") },
        modifier = Modifier.weight(1f)
    )
    Spacer(Modifier.width(8.dp))
    Button(onClick = { viewModel.addSkill() }) {
        Text("Add")
    }
}

// One row per skill, each with a Remove button
state.skills.forEach { skill ->
    Row(
        verticalAlignment = Alignment.CenterVertically,
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("• $skill", modifier = Modifier.weight(1f))
        TextButton(onClick = { viewModel.removeSkill(skill) }) {
            Text("Remove")
        }
    }
}

Spacer(Modifier.height(20.dp))
Button(
    onClick = { viewModel.showPreview() },
    modifier = Modifier.fillMaxWidth()
) {
    Text("Preview")
}
}
}

```

Step 5 — Build the preview and connect the screen

In the same `ProfileScreen.kt` file, add the read-only preview and a main screen that chooses which view to show:

```

@Composable
fun ProfilePreview(state: ProfileUiState, onBack: () -> Unit) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
            .verticalScroll(rememberScrollState())
    ) {
        Text("Profile Preview", fontSize = 24.sp,
            fontWeight = FontWeight.Bold)
        Spacer(Modifier.height(12.dp))

        Text("Name: ${state.name}")
        Text("Email: ${state.email}")
        Text("Contact: ${state.contactNumber}")
        Text("Address: ${state.address}")
    }
}

```

```

        Text("Username: ${state.username}")

        Spacer(Modifier.height(8.dp))
        Text("Skills:", fontWeight = FontWeight.Bold)
        if (state.skills.isEmpty()) {
            Text("No skills added yet.")
        } else {
            state.skills.forEach { skill -> Text("• $skill") }
        }

        Spacer(Modifier.height(20.dp))
        OutlinedButton(onClick = onBackPressed) {
            Text("Back to edit")
        }
    }
}

@Composable
fun ProfileScreen(viewModel: ProfileViewModel = viewModel()) {
    val state by viewModel.uiState.collectAsStateWithLifecycle()

    if (state.isPreview) {
        ProfilePreview(state = state, onBackPressed = { viewModel.backToEdit() })
    } else {
        ProfileForm(state = state, viewModel = viewModel)
    }
}

```

The `if (state.isPreview)` line is the whole trick: when the user taps Preview, the state changes and Compose automatically shows the other screen. No manual navigation needed.

Step 6 — Show the screen and run

Open `MainActivity.kt` and make `setContent` display your screen:

```

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                Surface {
                    ProfileScreen()
                }
            }
        }
    }
}

```

1. Press the green Run ► button and choose your emulator or device.

2. Fill in every field, add at least 2 skills, and remove one to test that removal works.
3. Tap **Preview** to see your profile, then **Back to edit**. Rotate the screen — your data should NOT disappear.

6. Expected Output

When your app runs it should have:

- An editable form with fields: Full name, Email, Contact number, Address, and Username.
- An “Add a skill” field with an **Add** button; each added skill appears in a list with a **Remove** button.
- A **Preview** button that opens a read-only screen showing all entered info and skills, with a **Back to edit** button.

7. Screenshot Requirement

You must submit a screenshot of your running application.

1. Run the app, fill in the profile, and add at least 2 skills.
2. Tap **Preview**, then capture the screen (emulator toolbar camera icon, or your OS screenshot shortcut). Save it as `screenshot.png`.
3. Put the image in a `/screenshots` folder in your project, then commit it with your code (see Section 8).

Your screenshot must clearly show

the Preview screen with your filled-in profile fields and at least 2 skills, from the app actually running on an emulator or device (not a photo of code). You may include a second screenshot of the edit form.

8. Submission Guidelines

Confirm one last time that you are on the **lab-activity-8** branch, then commit and push everything:

```
# Double-check the branch (must show * lab-activity-8)
git branch

# Stage everything, including your screenshot
git add .

# Commit with a clear message
git commit -m "Lab Activity 8: MVVM user profile app"

# Push the branch to your remote repository
git push -u origin lab-activity-8
```

Turn in the following (as your instructor directs):

- Your **code**, committed and pushed on the `lab-activity-8` branch.
- The **screenshot** of the running app (committed in `/screenshots`).
- The **repository link** (if pushing to GitHub) or a zip of the project, per your instructor.

9. Grading Rubric (100 Points)

Criteria	Points	What full marks looks like
Correct branch & submission	10	All work is committed and pushed on the lab-activity-8 branch (current branch shows lab-activity-8). Code is present in the repository.
App builds and runs	15	The project compiles in Android Studio and launches on an emulator or device without crashing.
UI state (data class)	10	A ProfileUiState data class holds all fields (name, email, contact, address, username) plus a skills list, with sensible defaults.
ViewModel field updates	15	ProfileViewModel extends ViewModel, exposes read-only StateFlow via a private backing property, and updates each field immutably with copy().
Add & remove skills	15	Users can type a skill and add it to the list, and remove any skill. The list is updated immutably (no in-place editing).
Preview feature	15	A Preview button switches to a read-only view showing all entered profile info and skills; a way to return to editing exists.
Compose UI wiring	10	Text fields show state and send changes to the ViewModel (value + onChange). No business logic in the UI.
Screenshot of running app	5	A clear screenshot of the Preview screen with a filled-in profile and at least 2 skills is committed.
Code quality	5	Readable names, correct file/package placement, no unused code, and consistent formatting.
TOTAL		100 points

10. Troubleshooting (Common Errors)

- **Typing does nothing / text won't appear** — each field needs BOTH `value = state.field` and `onValueChange = { viewModel.onXChange(it) }`.
- **Skills don't add** — make sure the Add button calls `viewModel.addSkill()` and that `addSkill()` uses `copy(skills = skills + skill)`.

- **Remove doesn't work** — use `current.copy(skills = current.skills - skill)`; never edit the list in place.
- **Form is cut off at the bottom** — add `.verticalScroll(rememberScrollState())` to the Column's modifier.
- **"Unresolved reference: viewModel / collectAsStateWithLifecycle"** — add the two Step 1 dependencies and click Sync.
- **Red import errors** — click the red text and press **Alt+Enter** (Option+Enter on Mac).
- **Work not graded** — you committed on the wrong branch. Run `git branch` and make sure `lab-activity-8` has the `*`.

11. Bonus Challenge (Optional, +5)

Only attempt after the required app works and is committed.

- Disable the **Preview** button until name and email are filled (hint: `enabled = state.name.isNotBlank() && state.email.isNotBlank()`).
- Prevent adding a skill that is already in the list.
- Add a **Clear all** button that resets the form to an empty `ProfileUiState()`.

12. Before You Submit — Quick Checklist

- `git branch` shows `* lab-activity-8`.
- App builds and runs without crashing.
- All five fields update as you type; you can add and remove skills.
- The Preview button shows the profile, and Back to edit returns to the form.
- Screenshot of the Preview screen (with 2+ skills) is added and committed.
- Everything is committed and pushed on `lab-activity-8`.